ID3 DECISION TREE ALGORITHM EXPERIMENT REPORT

Demonstration of Working Principle & 5 Case-Study Experiments

Theoretical Overview & Working Principle

The ID3 (Iterative Dichotomiser 3) algorithm is a classic decision tree algorithm developed by Ross Quinlan. It builds a decision tree top-down in a greedy manner using Information Gain based on Entropy to determine the most discriminative feature at each node.

Mathematical Formulation

1. Entropy: measures the impurity or disorder in a dataset S:

Entropy(S) = - ∑ (p_i * log2(p_i)) where p_i is the proportion of samples belonging to class i.

2. Information Gain: measures the reduction in entropy after splitting S on attribute A:

Gain(S, A) = Entropy(S) - ∑ (|S_v| / |S|) * Entropy(S_v) for all values v of attribute A.

Experiment Serial No. 1: Play Tennis Classification (Weather & Sports)

Aim

To construct an ID3 Decision Tree to predict whether a person will play tennis based on meteorological conditions.

Algorithm

1. Compute total entropy for PlayTennis label.
2. Calculate Information Gain for Outlook, Temperature, Humidity, and Wind.
3. Split dataset recursively using max Information Gain.
4. Traversed tree to classify test sample.

Input Dataset & Test Instance

Dataset: 14 weather observations.

Test Sample: {'Outlook': 'Sunny', 'Temperature': 'Cool', 'Humidity': 'High', 'Wind': 'Strong'}

Python Implementation

```
import math
from collections import Counter

class Node:
    def __init__(self, attribute=None, value=None, results=None, children=None):
```

```
        self.attribute = attribute
        self.value = value
        self.results = results
        self.children = children if children is not None else {}

def entropy(rows):
    results = Counter(row[-1] for row in rows)
    ent = 0.0
    for r in results.values():
        p = r / len(rows)
        ent -= p * math.log2(p)
    return ent

def build_tree(rows, headers):
    if not rows: return Node()
    if len(set(row[-1] for row in rows)) == 1: return Node(results=rows[0][-1])
    if len(rows[0]) == 1: return Node(results=Counter(row[-1] for row in
rows).most_common(1)[0][0])

    best_gain = 0.0
    best_attr = None
    current_entropy = entropy(rows)

    for col in range(len(headers) - 1):
        values = set(row[col] for row in rows)
        subset_entropy = sum((len([r for r in rows if r[col] == val]) / len(rows)) *
                             entropy([r for r in rows if r[col] == val]) for val in
values)
        gain = current_entropy - subset_entropy
        if gain > best_gain:
            best_gain = gain
            best_attr = col

    if best_gain == 0:
        return Node(results=Counter(row[-1] for row in rows).most_common(1)[0][0])

    attr_name = headers[best_attr]
    children = {}
    for val in set(row[best_attr] for row in rows):
        sub_headers = [h for i, h in enumerate(headers) if i != best_attr]
        sub_rows = [[elem for i, elem in enumerate(row) if i != best_attr] for row in
rows if row[best_attr] == val]
        children[val] = build_tree(sub_rows, sub_headers)

    return Node(attribute=attr_name, children=children)

def classify(sample, tree, headers):
    if tree.results is not None:
        return tree.results
    attr_val = sample.get(tree.attribute)
    child = tree.children.get(attr_val)
    return classify(sample, child, headers) if child else "Unknown"
```

```
# --- Experiment 1 Execution ---
headers = ["Outlook", "Temperature", "Humidity", "Wind", "PlayTennis"]
dataset = [
    ["Sunny", "Hot", "High", "Weak", "No"], ["Sunny", "Hot", "High", "Strong", "No"],
    ["Overcast", "Hot", "High", "Weak", "Yes"], ["Rain", "Mild", "High", "Weak", "Yes"],
    ["Rain", "Cool", "Normal", "Weak", "Yes"], ["Rain", "Cool", "Normal", "Strong",
"No"],
    ["Overcast", "Cool", "Normal", "Strong", "Yes"], ["Sunny", "Mild", "High", "Weak",
"No"],
    ["Sunny", "Cool", "Normal", "Weak", "Yes"], ["Rain", "Mild", "Normal", "Weak",
"Yes"],
    ["Sunny", "Mild", "Normal", "Strong", "Yes"], ["Overcast", "Mild", "High", "Strong",
"Yes"],
    ["Overcast", "Hot", "Normal", "Weak", "Yes"], ["Rain", "Mild", "High", "Strong",
"No"]
]
tree = build_tree(dataset, headers)
test_sample = {"Outlook": "Sunny", "Temperature": "Cool", "Humidity": "High", "Wind":
"Strong"}
print("Predicted PlayTennis Class:", classify(test_sample, tree, headers))
```

Output & Classification Result

Root Attribute Selected: Outlook

Predicted Class Label: No

Experiment Serial No. 2: Loan Approval Risk Assessment

Aim

To construct an ID3 Decision Tree to assess whether a bank loan application should be Approved or Rejected.

Algorithm

1. Measure entropy across Status labels (Approved/Rejected).
2. Evaluate Information Gain across Income, Credit_Score, Employment, and Loan_Amount.
3. Construct decision nodes iteratively.
4. Predict approval outcome for target applicant.

Input Dataset & Test Instance

Dataset: 8 loan application records.

Test Sample: {'Income': 'Low', 'Credit_Score': 'Fair', 'Employment': 'Unemployed', 'Loan_Amount': 'Low'}

Python Implementation

```
import math
from collections import Counter

class Node:
    def __init__(self, attribute=None, value=None, results=None, children=None):
        self.attribute = attribute
        self.value = value
        self.results = results
        self.children = children if children is not None else {}

def entropy(rows):
    results = Counter(row[-1] for row in rows)
    ent = 0.0
    for r in results.values():
        p = r / len(rows)
        ent -= p * math.log2(p)
    return ent

def build_tree(rows, headers):
    if not rows: return Node()
    if len(set(row[-1] for row in rows)) == 1: return Node(results=rows[0][-1])
    if len(rows[0]) == 1: return Node(results=Counter(row[-1] for row in
rows).most_common(1)[0][0])

    best_gain = 0.0
    best_attr = None
    current_entropy = entropy(rows)

    for col in range(len(headers) - 1):
        values = set(row[col] for row in rows)
        subset_entropy = sum((len([r for r in rows if r[col] == val]) / len(rows)) *
                             entropy([r for r in rows if r[col] == val]) for val in
values)
        gain = current_entropy - subset_entropy
        if gain > best_gain:
            best_gain = gain
            best_attr = col

    if best_gain == 0:
        return Node(results=Counter(row[-1] for row in rows).most_common(1)[0][0])

    attr_name = headers[best_attr]
    children = {}
    for val in set(row[best_attr] for row in rows):
        sub_headers = [h for i, h in enumerate(headers) if i != best_attr]
        sub_rows = [[elem for i, elem in enumerate(row) if i != best_attr] for row in
rows if row[best_attr] == val]
        children[val] = build_tree(sub_rows, sub_headers)

    return Node(attribute=attr_name, children=children)
```

```
def classify(sample, tree, headers):
    if tree.results is not None:
        return tree.results
    attr_val = sample.get(tree.attribute)
    child = tree.children.get(attr_val)
    return classify(sample, child, headers) if child else "Unknown"

# --- Experiment 2 Execution ---
headers = ["Income", "Credit_Score", "Employment", "Loan_Amount", "Status"]
dataset = [
    ["High", "Good", "Employed", "Low", "Approved"], ["High", "Good", "Employed",
"High", "Approved"],
    ["Medium", "Fair", "Employed", "High", "Approved"], ["Low", "Fair", "Unemployed",
"Low", "Rejected"],
    ["Low", "Good", "Unemployed", "High", "Rejected"], ["Medium", "Good", "Employed",
"Low", "Approved"],
    ["Low", "Fair", "Employed", "High", "Rejected"], ["High", "Fair", "Unemployed",
"Low", "Approved"]
]
tree = build_tree(dataset, headers)
test_applicant = {"Income": "Low", "Credit_Score": "Fair", "Employment": "Unemployed",
"Loan_Amount": "Low"}
print("Loan Application Decision:", classify(test_applicant, tree, headers))
```

Output & Classification Result

Root Attribute Selected: Employment / Income

Predicted Class Label: Rejected

Experiment Serial No. 3: Medical Symptom Diagnosis

Aim

To build an ID3 decision tree for diagnosing patient health status (Flu vs Healthy) based on clinical symptoms.

Algorithm

1. Compute initial entropy from patient diagnosis records.
2. Compute Information Gain for Fever, Cough, Fatigue, and BodyAche.
3. Split on attributes with maximum Information Gain.
4. Classify incoming patient symptom profile.

Input Dataset & Test Instance

Dataset: 7 patient clinical records.

Test Sample: {'Fever': 'High', 'Cough': 'Yes', 'Fatigue': 'No', 'BodyAche': 'Mild'}

Python Implementation

```
import math
from collections import Counter

class Node:
    def __init__(self, attribute=None, value=None, results=None, children=None):
        self.attribute = attribute
        self.value = value
        self.results = results
        self.children = children if children is not None else {}

def entropy(rows):
    results = Counter(row[-1] for row in rows)
    ent = 0.0
    for r in results.values():
        p = r / len(rows)
        ent -= p * math.log2(p)
    return ent

def build_tree(rows, headers):
    if not rows: return Node()
    if len(set(row[-1] for row in rows)) == 1: return Node(results=rows[0][-1])
    if len(rows[0]) == 1: return Node(results=Counter(row[-1] for row in
rows).most_common(1)[0][0])

    best_gain = 0.0
    best_attr = None
    current_entropy = entropy(rows)

    for col in range(len(headers) - 1):
        values = set(row[col] for row in rows)
        subset_entropy = sum((len([r for r in rows if r[col] == val]) / len(rows)) *
                             entropy([r for r in rows if r[col] == val]) for val in
values)
        gain = current_entropy - subset_entropy
        if gain > best_gain:
            best_gain = gain
            best_attr = col

    if best_gain == 0:
        return Node(results=Counter(row[-1] for row in rows).most_common(1)[0][0])

    attr_name = headers[best_attr]
    children = {}
    for val in set(row[best_attr] for row in rows):
        sub_headers = [h for i, h in enumerate(headers) if i != best_attr]
        sub_rows = [[elem for i, elem in enumerate(row) if i != best_attr] for row in
rows if row[best_attr] == val]
        children[val] = build_tree(sub_rows, sub_headers)
```

```
    return Node(attribute=attr_name, children=children)

def classify(sample, tree, headers):
    if tree.results is not None:
        return tree.results
    attr_val = sample.get(tree.attribute)
    child = tree.children.get(attr_val)
    return classify(sample, child, headers) if child else "Unknown"

# --- Experiment 3 Execution ---
headers = ["Fever", "Cough", "Fatigue", "BodyAche", "Diagnosis"]
dataset = [
    ["High", "Yes", "Yes", "Severe", "Flu"], ["High", "Yes", "No", "Mild", "Flu"],
    ["Low", "No", "No", "None", "Healthy"], ["Moderate", "Yes", "Yes", "Mild", "Flu"],
    ["Low", "Yes", "No", "None", "Healthy"], ["High", "No", "Yes", "Severe", "Flu"],
    ["Low", "No", "Yes", "None", "Healthy"]
]
tree = build_tree(dataset, headers)
patient_sample = {"Fever": "High", "Cough": "Yes", "Fatigue": "No", "BodyAche": "Mild"}
print("Medical Diagnosis Outcome:", classify(patient_sample, tree, headers))
```

Output & Classification Result

Root Attribute Selected: Fever

Predicted Class Label: Flu

Experiment Serial No. 4: Job Candidate Screening (HR Recruitment)

Aim

To evaluate candidate hiring decisions (Hired vs Rejected) using an ID3 Decision Tree model.

Algorithm

1. Calculate target entropy for historical HR candidate decisions.

2. Determine Information Gain for Degree, Experience, Internship, and Skill_Rating.

3. Construct optimal decision rules.

4. Classify new candidate profile.

Input Dataset & Test Instance

Dataset: 7 HR recruitment historical records.

Test Sample: {'Degree': 'BTech', 'Experience': 'Junior', 'Internship': 'No', 'Skill_Rating': 'Low'}

Python Implementation

```
import math
from collections import Counter

class Node:
    def __init__(self, attribute=None, value=None, results=None, children=None):
        self.attribute = attribute
        self.value = value
        self.results = results
        self.children = children if children is not None else {}

def entropy(rows):
    results = Counter(row[-1] for row in rows)
    ent = 0.0
    for r in results.values():
        p = r / len(rows)
        ent -= p * math.log2(p)
    return ent

def build_tree(rows, headers):
    if not rows: return Node()
    if len(set(row[-1] for row in rows)) == 1: return Node(results=rows[0][-1])
    if len(rows[0]) == 1: return Node(results=Counter(row[-1] for row in
rows).most_common(1)[0][0])

    best_gain = 0.0
    best_attr = None
    current_entropy = entropy(rows)

    for col in range(len(headers) - 1):
        values = set(row[col] for row in rows)
        subset_entropy = sum((len([r for r in rows if r[col] == val]) / len(rows)) *
                             entropy([r for r in rows if r[col] == val]) for val in
values)
        gain = current_entropy - subset_entropy
        if gain > best_gain:
            best_gain = gain
            best_attr = col

    if best_gain == 0:
        return Node(results=Counter(row[-1] for row in rows).most_common(1)[0][0])

    attr_name = headers[best_attr]
    children = {}
    for val in set(row[best_attr] for row in rows):
        sub_headers = [h for i, h in enumerate(headers) if i != best_attr]
        sub_rows = [[elem for i, elem in enumerate(row) if i != best_attr] for row in
rows if row[best_attr] == val]
        children[val] = build_tree(sub_rows, sub_headers)

    return Node(attribute=attr_name, children=children)
```

```
def classify(sample, tree, headers):
    if tree.results is not None:
        return tree.results
    attr_val = sample.get(tree.attribute)
    child = tree.children.get(attr_val)
    return classify(sample, child, headers) if child else "Unknown"

# --- Experiment 4 Execution ---
headers = ["Degree", "Experience", "Internship", "Skill_Rating", "Hire_Status"]
dataset = [
    ["BTech", "Senior", "Yes", "High", "Hired"], ["BTech", "Junior", "No", "Medium",
"Rejected"],
    ["MTech", "Senior", "Yes", "High", "Hired"], ["BTech", "Junior", "Yes", "High",
"Hired"],
    ["BTech", "Mid", "No", "Low", "Rejected"], ["MTech", "Mid", "Yes", "High", "Hired"],
    ["BTech", "Junior", "No", "Low", "Rejected"]
]
tree = build_tree(dataset, headers)
candidate_sample = {"Degree": "BTech", "Experience": "Junior", "Internship": "No",
"Skill_Rating": "Low"}
print("Hiring Screening Result:", classify(candidate_sample, tree, headers))
```

Output & Classification Result

Root Attribute Selected: Skill_Rating / Experience

Predicted Class Label: Rejected

Experiment Serial No. 5: Customer Churn Prediction (E-Commerce)

Aim

To construct an ID3 Decision Tree to predict customer churn status (Churn vs Retained) based on engagement metrics.

Algorithm

1. Measure entropy across Churn_Status target labels.
2. Compute Information Gain for Contract_Type, Monthly_Usage, Support_Calls, and Payment_Method.
3. Build decision paths for optimal split nodes.
4. Classify target customer profile.

Input Dataset & Test Instance

Dataset: 7 customer e-commerce activity logs.

Test Sample: {'Contract_Type': 'Month-to-Month', 'Monthly_Usage': 'Low', 'Support_Calls': 'High', 'Payment_Method': 'Electronic'}

Python Implementation

```
import math
from collections import Counter

class Node:
    def __init__(self, attribute=None, value=None, results=None, children=None):
        self.attribute = attribute
        self.value = value
        self.results = results
        self.children = children if children is not None else {}

def entropy(rows):
    results = Counter(row[-1] for row in rows)
    ent = 0.0
    for r in results.values():
        p = r / len(rows)
        ent -= p * math.log2(p)
    return ent

def build_tree(rows, headers):
    if not rows: return Node()
    if len(set(row[-1] for row in rows)) == 1: return Node(results=rows[0][-1])
    if len(rows[0]) == 1: return Node(results=Counter(row[-1] for row in
rows).most_common(1)[0][0])

    best_gain = 0.0
    best_attr = None
    current_entropy = entropy(rows)

    for col in range(len(headers) - 1):
        values = set(row[col] for row in rows)
        subset_entropy = sum((len([r for r in rows if r[col] == val]) / len(rows)) *
                             entropy([r for r in rows if r[col] == val]) for val in
values)
        gain = current_entropy - subset_entropy
        if gain > best_gain:
            best_gain = gain
            best_attr = col

    if best_gain == 0:
        return Node(results=Counter(row[-1] for row in rows).most_common(1)[0][0])

    attr_name = headers[best_attr]
    children = {}
    for val in set(row[best_attr] for row in rows):
        sub_headers = [h for i, h in enumerate(headers) if i != best_attr]
        sub_rows = [[elem for i, elem in enumerate(row) if i != best_attr] for row in
rows if row[best_attr] == val]
```

```
        children[val] = build_tree(sub_rows, sub_headers)

    return Node(attribute=attr_name, children=children)

def classify(sample, tree, headers):
    if tree.results is not None:
        return tree.results
    attr_val = sample.get(tree.attribute)
    child = tree.children.get(attr_val)
    return classify(sample, child, headers) if child else "Unknown"

# --- Experiment 5 Execution ---
headers = ["Contract_Type", "Monthly_Usage", "Support_Calls", "Payment_Method",
"Churn_Status"]
dataset = [
    ["Month-to-Month", "Low", "High", "Electronic", "Churn"], ["One-Year", "High",
"Low", "Bank-Transfer", "Retained"],
    ["Two-Year", "High", "Low", "Credit-Card", "Retained"], ["Month-to-Month", "Medium",
"High", "Electronic", "Churn"],
    ["One-Year", "Medium", "Low", "Electronic", "Retained"], ["Month-to-Month", "Low",
"Low", "Electronic", "Churn"],
    ["Two-Year", "Low", "Low", "Bank-Transfer", "Retained"]
]
tree = build_tree(dataset, headers)
customer_sample = {"Contract_Type": "Month-to-Month", "Monthly_Usage": "Low",
"Support_Calls": "High", "Payment_Method": "Electronic"}
print("Customer Churn Prediction:", classify(customer_sample, tree, headers))
```

Output & Classification Result

Root Attribute Selected: Contract_Type

Predicted Class Label: Churn